ECE 650 – Assignment 1

Street Graph Builder: Problem, Approach and Code Walkthrough

Fall 2026 · Python 3 · main file: a1/ece650-a1.py

Section	What it covers
1. The assignment	What the program has to do and the rules it must follow
2. CS background	The computer-science ideas the code relies on, from first principles
3. The approach	How the solution is structured and why
4. Code walkthrough	Every block of the source file, explained in order
5. Worked example	The sample input traced through the algorithm
6. Assumptions	Ambiguities in the spec and how they were resolved
7. Testing	How the program was checked
8. Alternatives	Other approaches that could have been taken, and trade-offs

1. The assignment

The project is to help a police department decide where to put traffic cameras. Later assignments solve the **vertex cover** problem on a graph; this first assignment builds that graph from a description of streets.

1.1 Input

The program reads commands from standard input, one per line:

Command	Meaning
`add "Name" (x1,y1) (x2,y2) ...`	Add a street. The points form a poly-line: a segment from each point to the next. Coordinates are integers.
`mod "Name" (x1,y1) ...`	Replace the points of an existing street.
`rm "Name"`	Remove an existing street.
`gg`	Print the current graph.

1.2 The graph to output

Vertices are (a) every point where two different streets intersect, and (b) both end-points of any street segment that takes part in an intersection.

Edges join two vertices if and only if: at least one of them is an intersection, both lie on the same street, and one can be reached from the other along that street without passing through another vertex.

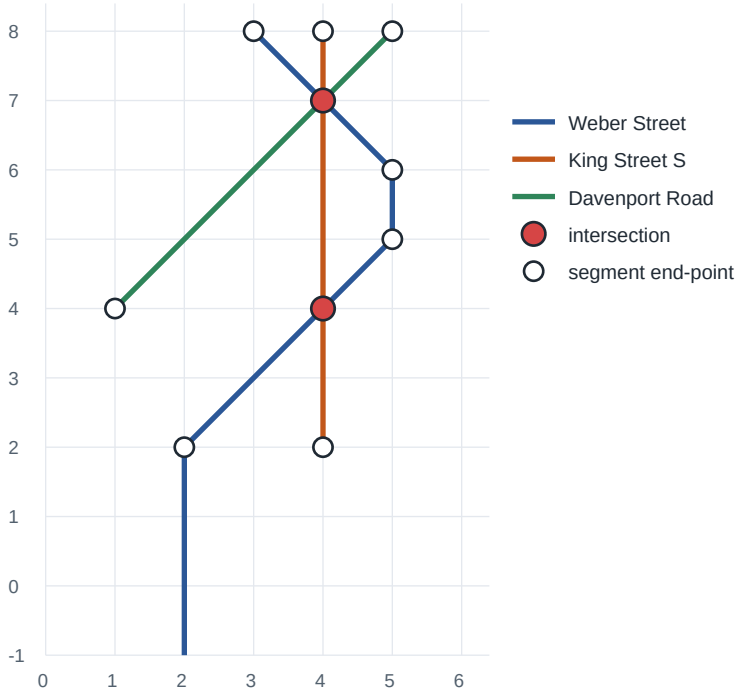

Figure 1. The sample input. The red points are intersections; white points are end-points of segments that touch an intersection. (5,5)–(5,6) is a segment between two vertices that gets no edge, because neither end is an intersection.

Output format:

```
V = {
  1: (2,2)
  2: (4,2)
  ...
}
E = {
  <1,3>,
  <2,3>,
  ...
  <6,10>
}
```

1.3 Errors and other rules

- Any incorrect line prints a message starting with **Error:** to **standard error**; the line is ignored and the program keeps running. It must never crash.
- Intersection coordinates may not be integers even though inputs are.
- The amount of whitespace between components does not matter.
- Output is graded automatically, so the format must match exactly.
- Only libraries already installed on the department servers may be used, so the solution uses only the Python standard library.

2. CS background

This section explains the general computer-science ideas the program is built on. Each part ends with a note on where it shows up in the code. If a topic is already familiar, skip it.

2.1 Standard streams, pipes and end of file

Every program starts with three open "streams": **standard input** (stdin) for reading, **standard output** (stdout) for normal output, and **standard error** (stderr) for error messages. By default all three are connected to the terminal, but the shell can redirect them: `prog < file.txt` feeds a file to stdin, `prog 2>/dev/null` throws away stderr, and `a | b` (a **pipe**) connects a's stdout to b's stdin.

Keeping errors on stderr means anything reading stdout, such as the marker's script or another program in a pipe, only sees the graph. When a stream has nothing left to give, it is at **end of file** (EOF). In a terminal, Ctrl+D sends EOF.

Output is often **buffered**: the program collects text in memory and writes it out in chunks. That's efficient, but a program on the other end of a pipe may wait for output that is still sitting in the buffer. **Flushing** forces it out immediately.

> **In this program:** `sys.stdin.readline()` reads one line and returns `""` at EOF, which ends the main loop. Errors use `print(..., file=sys.stderr)`, and every print uses `flush=True`.

2.2 Strings and parsing

Parsing means turning raw text into structured data the program can work with, and rejecting text that doesn't follow the expected format. A common way to do it by hand is a **scanner**: keep a position in the string and move forward piece by piece, checking that each piece is what you expect.

Python strings are sequences of characters. The operations used here:

Operation	Example	Result
`s[i]`	`"add"[0]`	`"a"` – one character (indexing starts at 0)
`s[i:j]`	`"(4,2)"[1:4]`	`"4,2"` – a **slice**, from i up to but not including j
`s.strip()`	`" gg ".strip()`	`"gg"` – remove whitespace at both ends (`lstrip` only the left)
`s.find(c, start)`	`'"A B" (1,2)'.find('"', 1)`	4 – index of the next c, or –1 if there is none
`s.split(sep)`	`"4, 2".split(",")`	`['4', ' 2']`
`s.split()`	`"Main St".split()`	`['Main', 'St']` – splits on any run of whitespace
`' '.join(list)`	`' '.join(['Main','St'])`	`"Main St"`
`c.isspace()`	`" ".isspace()`	`True`

In this program: `split_command`, `read_name` and `parse_points` are small scanners. Each one checks a piece of the line, then hands the rest of the line to the next step.

2.3 Lists, tuples, dictionaries and sets

Type	Looks like	Key property	Used for
list	`[1, 2, 3]`	Ordered, can change	streets' points, segments, edges
tuple	`(4, 2)`	Ordered, cannot change (**immutable**)	points (x, y)
dict	`{key: value}`	Look up a value by key	streets by name; vertex ID by point
set	`{a, b}`	No duplicates; fast "is x in it?"	intersections; edges already added
frozenset	`frozenset((p, q))`	An immutable set	an undirected edge, where {p, q} = {q, p}

Dictionaries and sets are built on **hash tables**. Python runs a hash function on the key to get a number that says where to store it, so finding a key takes about the same time no matter how many items there are (O(1) on average). Only values that can't change can be hashed: if a key changed after it was stored, its hash would change and it would never be found again. That's why points are tuples, not lists, and why an edge is a frozenset rather than a set.

In this program: A point such as `(Fraction(4), Fraction(7))` is used directly as a dictionary key in `self.ids` to remember its vertex number.

2.4 Exceptions

An **exception** is a way to stop the current work and jump straight to code that handles the problem. `raise` throws one; a `try/except` block further up catches it. Code in between doesn't need to check for errors after every step. A program can define its own exception types by subclassing `Exception`.

In this program: Any validation step anywhere raises `InputError`. The single `try/except` in `main` prints the message and moves to the next line, so a bad line can never crash the program.

2.5 Classes and objects

A **class** bundles data (attributes) with functions that work on that data (methods). An **object** is one instance of a class. `__init__` runs when the object is created and sets up its attributes; `self` refers to the object the method was called on.

In this program: `StreetDB` holds the streets and the vertex-number table, and its methods `add`, `mod`, `rm` and `build_graph` are the only code that changes or reads them.

2.6 How computers store numbers

Python `int` values are exact and can be any size. `float` values are stored in binary with about 16 significant digits, so most decimals can't be stored exactly: in Python, `0.1 + 0.2 == 0.3` is `False`. For geometry this means two calculations of the same intersection can give slightly different floats,

and the program would think they are two different points.

A **rational number** is a fraction of two integers, such as 3/2. Python's `Fraction` stores the numerator and denominator as exact integers. Adding, subtracting, multiplying and dividing rational numbers always gives another rational number, so there's never any rounding.

> **In this program:** Every intersection of two segments with integer end-points has rational coordinates, so using `Fraction` makes all geometry exact. Values are only rounded at the very end, for printing.

2.7 Vector geometry

A **vector** is a direction and length, written (dx, dy). The vector from point a to point b is $b - a = (b_x - a_x, b_y - a_y)$. Three operations do all the work in this program:

- **Cross product** (2D): $u \times v = u_x v_y - u_y v_x$. For vectors o→a and o→b, its sign says which side of the line oa the point b is on: positive means left, negative means right, and **zero means the three points are on one line** (collinear).
- **Dot product**: $u \cdot v = u_x v_x + u_y v_y$. For a point p on a segment a→b, $(p - a) \cdot (b - a)$ grows steadily from a to b, so it can be used to sort points along the segment.
- **Parametric form**: every point on segment p1→p2 can be written p1 + t(p2 – p1) for some t between 0 and 1. t = 0 is p1, t = 1 is p2, t = 0.5 is the midpoint. To intersect two segments, set their parametric forms equal and solve the two resulting equations for t and u (**Cramer's rule**, a standard formula for solving two linear equations). The segments meet if both t and u lie in [0, 1].

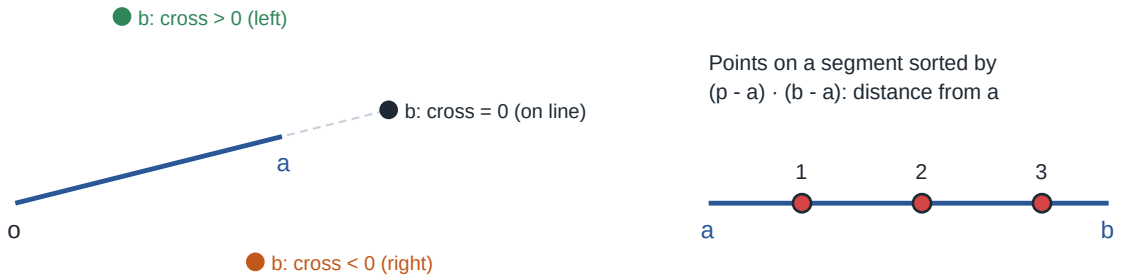

Figure 2. Left: the sign of the cross product tells which side of a line a point is on. Right: the dot product orders points along a segment.

> **In this program:** `cross` and `on_segment` use the cross product, `intersect` uses the parametric form, and `build_graph` sorts with the dot product.

2.8 Graphs

In computer science a **graph** is a set of **vertices** (also called nodes) joined by **edges**. In an **undirected** graph an edge has no direction: <u,v> and <v,u> are the same edge. Graphs model anything with connections: road networks, social networks, computer networks.

A **vertex cover** is a set of vertices that touches every edge. Here, the vertices are intersections and street points and the edges are stretches of road. A camera at each vertex in a cover watches every stretch. Finding the *smallest* cover is NP-hard: no known algorithm solves it quickly for every graph, which later assignments will explore.

In this program: This assignment builds the graph; `print_graph` outputs the vertex set V and edge set E.

2.9 Algorithm cost (Big-O)

Big-O notation describes how running time grows as the input grows, ignoring constant factors. O(n) means time grows in step with n; $O(n^2)$ means doubling the input roughly quadruples the time. Checking every pair of n items is $O(n^2)$, because there are $n(n - 1)/2$ pairs.

In this program: With n segments and V vertices, finding intersections is $O(n^2)$ and building edges is O(n·V). For the street counts in this course that is a fraction of a second.

2.10 Python features used

Feature	What it does
`for a, b in zip(xs, xs[1:])`	Walks over neighbouring pairs: (x0,x1), (x1,x2), …
`for i, x in enumerate(xs)`	Gives each item with its index.
`[f(x) for x in xs if cond]`	**List comprehension**: build a list in one expression.
`xs.sort(key=lambda p: ...)`	Sort by a computed value. A **lambda** is a small unnamed function.
`def inner(): ...`	A function defined inside another can use the outer function's variables.
`a, b = pair`	**Unpacking**: assign each part of a tuple to its own name.
`"{}: ({},{})".format(...)`	Fill the {} placeholders with values.
`x if cond else y`	Choose between two values in one expression.
`if __name__ == "__main__":`	Run `main()` only when the file is run directly, not when another file imports it.

3. The approach

The program is a loop that reads one command at a time and updates a small in-memory database of streets. The graph is **not** kept up to date as streets change; it is rebuilt from scratch on every **gg**. With the input sizes in this course that is fast, and it removes a whole class of bugs that would come from keeping a graph in sync with add, mod and rm.

Stage	What happens	Code
1. Parse	Split off the command word, then scan the quoted name and each (x,y) pair with plain string operations. Anything unexpected is an error.	`split_command, read_name, parse_points`
2. Validate	Check the street name, the number of points, integer coordinates and repeated points.	`normalize_name, parse_int`
3. Store	Keep each street's points in a dictionary keyed by lower-case name.	`StreetDB.add / mod / rm`
4. Intersect	On gg, test every pair of segments from different streets and collect intersection points and the segment end-points involved.	`intersect, build_graph`
5. Connect	For each segment, sort the vertices lying on it by distance along the segment and join neighbours if one of them is an intersection.	`build_graph`
6. Print	Number each vertex (numbers stay fixed between gg calls) and print V and E.	`print_graph`

3.1 Key decisions

- **Plain string parsing.** Each part of a line is checked by a short function using basic string methods. It is easy to follow, and each check gives its own specific error message, such as “missing closing quote”.
- **Exact arithmetic.** Intersections are computed with `Fraction` (see 2.6). Two calculations of the same point always give identical values, so points can be dictionary keys without any tolerance comparisons.
- **Edges from segments, not from streets.** Working one segment at a time makes condition (c), “reachable without traversing another vertex”, simple: once the vertices on a segment are sorted, only neighbours in that order can be joined.
- **Stable vertex IDs.** A point keeps its ID for the whole run, matching the sample in the assignment where vertex 6 is still 6 after the mod command.
- **Errors as exceptions.** All validation raises one exception type, caught in one place (see 2.4). A failed command never leaves the database half-updated.

4. Code walkthrough

The file has four parts — parsing, geometry, the graph, and output plus the main loop — explained here in the order they appear.

4.1 Imports

```
import sys
from fractions import Fraction
```

`sys` gives access to standard input and standard error (2.1). `Fraction` provides exact rational numbers (2.6). Nothing outside the standard library is needed.

4.2 Parsing

LETTERS and InputError

```
LETTERS = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"


class InputError(Exception):
    pass
```

LETTERS lists the characters allowed in a street name, apart from spaces. `InputError` is the program's own exception type for bad input (2.4). Because the main loop catches only this type, a real bug in the program would not be quietly reported as “bad input”.

normalize_name

Source: lines 17–25 of a1/ece650-a1.py

```
def normalize_name(name):
    """Validate a street name and return (key, display_name)."""
    if name.strip() == "":
        raise InputError("street name cannot be empty")
    for ch in name:
        if ch not in LETTERS and ch != " ":
            raise InputError("street name may only contain letters and spaces")
    display = " ".join(name.split())
    return display.lower(), display
```

Rejects a name that is empty or only spaces, then checks every character. It returns two forms: a **key** (lower-case with extra spaces collapsed) used to look up the street, and a **display** form. `" ".join(name.split())` collapses any run of spaces into one, so `"Weber  Street"` and `"weber street"` give the same key.

split_command

Source: lines 28–34 of a1/ece650-a1.py

```
def split_command(line):
    """Split a line into (command, rest). rest keeps its leading whitespace."""
    line = line.strip()
    i = 0
    while i < len(line) and not line[i].isspace():
        i += 1
    return line[:i], line[i:]
```

After stripping the line, it moves an index `i` forward until it hits whitespace or the end. Everything before `i` is the command word; everything from `i` on is the rest, *including its leading space*. Keeping that space lets the next step check that the command and the name really were separated, so `add"A" (1,1) (2,2)` is read as the unknown command `add"A"` and rejected.

read_name

Source: lines 37–47 of a1/ece650-a1.py

```
def read_name(rest):
    """rest looks like ' "Street Name" ...'. Return (name, text after the name)."""
    if rest == "" or not rest[0].isspace():
        raise InputError("expected a space before the street name")
    rest = rest.lstrip()
    if not rest.startswith('"'):
        raise InputError("street name must be in double quotes")
    close = rest.find('"', 1)
    if close == -1:
        raise InputError("missing closing quote on street name")
    return rest[1:close], rest[close + 1:]
```

Checks there is at least one space, skips the spaces, requires an opening quote, then uses `find` to locate the closing quote, starting the search at position 1 so it doesn't find the opening one. The text between the quotes is the name; whatever follows the closing quote is returned for the next step to parse.

parse_int

Source: lines 50–55 of a1/ece650-a1.py

```
def parse_int(text):
    text = text.strip()
    digits = text[1:] if text.startswith("-") else text
    if digits == "" or any(ch not in "0123456789" for ch in digits):
        raise InputError("coordinates must be integers")
    return int(text)
```

Checks a coordinate is a whole number with an optional leading minus sign and only the digits 0–9, then converts it. Checking first matters: calling `int()` on bad text raises `ValueError`, not `InputError`, which the main loop does not catch. The explicit digit list also rejects things `int()` would accept, such as `+5` or `1_000`.

parse_points

Source: lines 58–81 of a1/ece650-a1.py

```
def parse_points(text):
    """Parse ' (x1,y1) (x2,y2) ...' into a list of (x, y) integer tuples."""
    pts = []
    while text.strip() != "":
        if not text[0].isspace():
            raise InputError("coordinates must be separated by spaces")
        text = text.lstrip()
        if not text.startswith("("):
            raise InputError("expected '(' at the start of a coordinate")
        close = text.find(")")
        if close == -1:
            raise InputError("missing ')' in a coordinate")
        parts = text[1:close].split(",")
        if len(parts) != 2:
```

```
            raise InputError("a coordinate must look like (x,y)")
        pts.append((parse_int(parts[0]), parse_int(parts[1])))
        text = text[close + 1:]

    if len(pts) < 2:
        raise InputError("a street needs at least two coordinates")
    for i in range(len(pts) - 1):
        if pts[i] == pts[i + 1]:
            raise InputError("consecutive coordinates of a street must differ")
    return pts
```

A scanner over the text after the name. Each time round the loop it:

- requires at least one space before the next coordinate, then skips the spaces;
- requires `(` and finds the matching `)`;
- splits what's between them on the comma, which must give exactly two parts, and converts each with `parse_int`;
- cuts the coordinate off the front of the text and goes round again.

The loop ends when only whitespace is left. Then two more checks: a street needs at least two points to form a segment, and two equal points in a row would make a zero-length segment with no direction, which would break the sorting step later.

4.3 Geometry

cross and on_segment

```
def cross(o, a, b):
    return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0])


def on_segment(p, a, b):
    """True if point p lies on the closed segment ab."""
    if cross(a, b, p) != 0:
        return False
    return (min(a[0], b[0]) <= p[0] <= max(a[0], b[0])
            and min(a[1], b[1]) <= p[1] <= max(a[1], b[1]))
```

`cross(o, a, b)` is the cross product of o→a and o→b (2.7): zero exactly when the three points are on one line.

`on_segment` checks that p is on the infinite line through a and b (cross product zero) *and* inside the rectangle that has a and b as opposite corners. Together these mean p is on the segment itself, including its end-points.

intersect

Source: lines 100–121 of a1/ece650-a1.py

```
def intersect(p1, p2, p3, p4):
    """Return the set of intersection points of segments p1p2 and p3p4.

    For collinear overlapping segments, the endpoints of the overlap are
    returned.
    """
    x1, y1 = p1
    x2, y2 = p2
    x3, y3 = p3
```

```
    x4, y4 = p4
    denom = (x1 - x2) * (y3 - y4) - (y1 - y2) * (x3 - x4)
    if denom != 0:
        t = Fraction((x1 - x3) * (y3 - y4) - (y1 - y3) * (x3 - x4), denom)
        u = Fraction((x1 - x3) * (y1 - y2) - (y1 - y3) * (x1 - x2), denom)
        if 0 <= t <= 1 and 0 <= u <= 1:
            return {(x1 + t * (x2 - x1), y1 + t * (y2 - y1))}
        return set()
    # Parallel: intersect only if collinear and overlapping.
    if cross(p1, p2, p3) != 0:
        return set()
    return {p for p in (p1, p2, p3, p4)
            if on_segment(p, p1, p2) and on_segment(p, p3, p4)}
```

Each segment is written in parametric form (2.7): P(t) = p1 + t(p2 – p1) and Q(u) = p3 + u(p4 – p3). Setting P(t) = Q(u) and solving with Cramer's rule gives the formulas for t and u, both divided by `denom`.

- **denom ≠ 0**: the lines cross at exactly one point. It is on both segments only if 0 ≤ t ≤ 1 and 0 ≤ u ≤ 1. `Fraction` keeps t and u exact.
- **denom = 0**: the segments are parallel. If p3 is not on the line through p1 and p2, they never meet. Otherwise they lie on the same line and may overlap. The overlap is itself a segment whose two ends are always among the four original end-points, so the function returns whichever of p1, p2, p3, p4 lie on both segments.
- Returning a **set** covers every case the same way: empty (no intersection), one point, or two points (an overlap).

to_point

```
def to_point(p):
    return (Fraction(p[0]), Fraction(p[1]))
```

Converts input points to Fractions so that input points and computed intersection points have the same type. They are compared and used as dictionary keys together.

4.4 The street database and graph

StreetDB state

```
class StreetDB:
    def __init__(self):
        self.streets = {}      # key -> (display name, [points])
        self.ids = {}          # point -> vertex id (stable across gg calls)
        self.next_id = 1
```

- `streets` maps the lower-case name to the display name and list of points.
- `ids` remembers the vertex number assigned to each point. It is never cleared, so a point that stays a vertex keeps its number across gg commands.
- `next_id` is the next unused vertex number.

add, mod, rm

```
def add(self, name, pts):
        key, display = normalize_name(name)
        if key in self.streets:
            raise InputError("`add' specified for a street that already exists")
        self.streets[key] = (display, pts)

def mod(self, name, pts):
        key, display = normalize_name(name)
        if key not in self.streets:
            raise InputError("`mod' or `rm' specified for a street that does not exist")
        self.streets[key] = (display, pts)

def rm(self, name):
        key, _ = normalize_name(name)
        if key not in self.streets:
            raise InputError("`mod' or `rm' specified for a street that does not exist")
        del self.streets[key]
```

Each one normalises the name, checks whether the street exists, and either raises `InputError` or updates the dictionary. The check happens *before* the change, so a failed command leaves the database untouched.

vertex_id

```
def vertex_id(self, p):
        if p not in self.ids:
            self.ids[p] = self.next_id
            self.next_id += 1
        return self.ids[p]
```

Returns a point's number, giving it the next free number the first time the point is seen.

build_graph – the core algorithm

Source: lines 162–212 of a1/ece650-a1.py

```
def build_graph(self):
        # All segments, tagged with the street they belong to.
        segments = []
        for key, (_, pts) in self.streets.items():
            fpts = [to_point(p) for p in pts]
            for a, b in zip(fpts, fpts[1:]):
                segments.append((key, a, b))
```

```
        vertices = []        # in the order they were found
        inter_set, vert_set = set(), set()

        def add_vertex(p):
            if p not in vert_set:
                vert_set.add(p)
                vertices.append(p)

        for i in range(len(segments)):
            si, a, b = segments[i]
            for j in range(i + 1, len(segments)):
                sj, c, d = segments[j]
                if si == sj:
                    continue
                pts = intersect(a, b, c, d)
                if not pts:
                    continue
                for e in (a, b, c, d):
                    add_vertex(e)
                for p in sorted(pts):
                    add_vertex(p)
                    inter_set.add(p)

        for p in vertices:
            self.vertex_id(p)

        # Edges: along each segment, connect consecutive vertices if at
        # least one of the pair is an intersection.
        edges = []
        edge_set = set()
        for _, a, b in segments:
            on = [p for p in vertices if on_segment(p, a, b)]
            dx, dy = b[0] - a[0], b[1] - a[1]
            on.sort(key=lambda p: (p[0] - a[0]) * dx + (p[1] - a[1]) * dy)
            for p, q in zip(on, on[1:]):
                if p == q or (p not in inter_set and q not in inter_set):
                    continue
                k = frozenset((p, q))
                if k not in edge_set:
                    edge_set.add(k)
                    edges.append((self.ids[p], self.ids[q]))

        return sorted(vertices, key=lambda p: self.ids[p]), edges
```

Step 1 – collect segments. Every street is split into its segments using `zip(fpts, fpts[1:])` (2.10). Each segment is tagged with its street's key so segments of the same street can be skipped later.

Step 2 – find vertices. The double loop tests every pair of segments once (j starts at i + 1), skipping pairs from the same street, since a street crossing itself is not an intersection with *another* street. When a pair meets, the intersection points go into `inter_set` and all four end-points become vertices. The list `vertices` keeps the order points were found in; `vert_set` answers “already seen?” quickly (2.3). The small inner function `add_vertex` keeps these two in step.

Step 3 – number the vertices with `vertex_id`.

Step 4 – build edges. For each segment a→b, take every vertex on it and sort by the dot product (p – a)·(b – a), which orders them from a to b (2.7). Neighbours in that order can reach each other without passing another vertex, so they are joined, but only if at least one is an intersection. An edge is stored as a `frozenset` of its two points (2.3) so the same edge is never added twice.

The method returns the vertices sorted by number and the edges as pairs of numbers.

4.5 Output

```
def fmt_num(v):
    if v.denominator == 1:
        return str(v.numerator)
    return "{:.2f}".format(float(v))


def print_graph(db):
    vertices, edges = db.build_graph()
    out = ["V = {"]
    for p in vertices:
        out.append("  {}: ({},{})".format(db.ids[p], fmt_num(p[0]), fmt_num(p[1])))
    out.append("}")
    out.append("E = {")
    for i, (u, v) in enumerate(edges):
        out.append("  <{},{}>{}".format(u, v, "," if i < len(edges) - 1 else ""))
    out.append("}")
    print("\n".join(out), flush=True)
```

`fmt_num` prints whole numbers without a decimal point, so integer points look exactly like the sample (`(2,2)`); other values get two decimal places (`1.50`). `print_graph` builds all the lines first and prints them together. The comma is left off the last edge to match the sample, and `flush=True` sends the output immediately (2.1).

4.6 Main loop

```
def process(line, db):
    cmd, rest = split_command(line)
    if cmd == "":
        return
    if cmd == "gg":
        if rest.strip() != "":
            raise InputError("`gg' takes no arguments")
        print_graph(db)
    elif cmd == "rm":
        name, after = read_name(rest)
        if after.strip() != "":
            raise InputError("`rm' takes only a street name")
        db.rm(name)
    elif cmd == "add" or cmd == "mod":
        name, after = read_name(rest)
        pts = parse_points(after)
        if cmd == "add":
            db.add(name, pts)
        else:
            db.mod(name, pts)
    else:
        raise InputError("unknown command `" + cmd + "'")


def main():
    db = StreetDB()
    while True:
        line = sys.stdin.readline()
        if line == "":
            break
        try:
            process(line, db)
```

```
        except InputError as e:
            print("Error: " + str(e), file=sys.stderr, flush=True)
    sys.exit(0)
```

`process` handles one line. It splits off the command word and uses an `if/elif` chain to pick what to do. A blank line gives an empty command and is ignored. `gg` and `rm` check that nothing unexpected follows. Any other word is an unknown command.

`main` reads lines until end of input (`readline` returns `""`) and wraps each in `try/except InputError`, so a bad line prints an error and the loop continues. Reading with `readline` rather than `for line in sys.stdin` avoids input-buffering delays when the program is driven through a pipe.

5. Worked example

Take the first three commands of the sample (Figure 1):

```
add "Weber Street" (2,-1) (2,2) (5,5) (5,6) (3,8)
add "King Street S" (4,2) (4,8)
add "Davenport Road" (1,4) (5,8)
```

Parsing one line

For `add "King Street S" (4,2) (4,8)`:

- `split_command` → command `"add"`, rest `' "King Street S" (4,2) (4,8)'`
- `read_name` → name `"King Street S"`, rest `' (4,2) (4,8)'`
- `parse_points` → `[(4, 2), (4, 8)]`
- `normalize_name` → key `"king street s"`, stored in the dictionary.

Finding intersections on gg

Segment pair	Result
Weber (2,2)–(5,5) × King (4,2)–(4,8)	cross at (4,4)
Weber (5,6)–(3,8) × King (4,2)–(4,8)	cross at (4,7)
Weber (5,6)–(3,8) × Davenport (1,4)–(5,8)	cross at (4,7)
King (4,2)–(4,8) × Davenport (1,4)–(5,8)	cross at (4,7)
All other pairs	no intersection

Worked through for the first row: p1 = (2,2), p2 = (5,5), p3 = (4,2), p4 = (4,8). denom = (2–5)(2–8) – (2–5)(4–4) = 18. t = [(2–4)(2–8) – (2–2)(4–4)] / 18 = 12/18 = 2/3, and u = 1/3. Both are between 0 and 1, so the point is (2,2) + 2/3·(3,3) = (4,4).

Intersections: (4,4) and (4,7). End-points of the segments involved: (2,2), (5,5), (5,6), (3,8), (4,2), (4,8), (1,4), (5,8). That is 10 vertices, matching the sample.

Building edges

- Weber (2,2)–(5,5): vertices in order (2,2), (4,4), (5,5) → two edges through (4,4).
- Weber (5,5)–(5,6): both ends are vertices but neither is an intersection → no edge. This is why condition (a) exists.
- Weber (5,6)–(3,8): (5,6), (4,7), (3,8) → two edges.
- King (4,2)–(4,8): (4,2), (4,4), (4,7), (4,8) → three edges, including (4,4)–(4,7) between two intersections.
- Davenport (1,4)–(5,8): (1,4), (4,7), (5,8) → two edges.

Total: 9 edges, matching the sample. After `mod "Weber Street" (2,1) (2,2)`, Weber no longer reaches King or Davenport, so only (4,7) remains as an intersection, giving 5 vertices and 4 edges. After removing King nothing intersects and the graph is empty.

6. Assumptions about ambiguous requirements

The specification says ambiguities should be clarified on Piazza. These are the interpretations used; each is a small change if Piazza says otherwise. Every choice is checked by at least one test in `tests/test_a1.py` (groups 1, 3 and 4 of its output); the last column names those tests.

Question	Choice made	Checked by
Are street names case-sensitive?	No. "Weber Street" and "weber street" are the same street.	add of the same name in other case is rejected; mod and rm work in any case
Does the amount of space inside a name matter?	No. Extra and trailing spaces are collapsed: "Weber Street " is "Weber Street".	"Weber Street" = "weber street"; rm "A " removes "A"
Does a street crossing itself create a vertex?	No. Only intersections between different streets count.	self-crossing street alone gives an empty graph; the crossing point stays out when another street crosses
What happens to a vertex number when the vertex disappears?	It is not reused. If the same point becomes a vertex again it gets its old number back.	numbers kept after mod (sample); old numbers return after rm and re-add; new vertices get new numbers
How are overlapping (collinear) streets handled?	The ends of the overlapping part are treated as intersections.	partial overlap; one street inside another; diagonal overlap; collinear without overlap gives nothing
How are coordinates printed?	Whole numbers as they are, (2,2); others rounded to two decimals, (1.50,1.50).	2/3 prints 0.67; negative –2/3 prints –0.67; mixed (2,0.50)
Is a space required between components, e.g. add "A"(1,2)?	Yes, between command, name and each point. Spaces inside brackets are optional; tabs count as spaces.	three missing-space errors; spaces inside brackets and tabs accepted
What characters may a street name contain?	Letters and spaces only.	"King Street S" accepted; digits and punctuation rejected
May a street return to an earlier point?	Yes, unless it is the point just before it, which would make a zero-length segment.	(0,4) (4,0) (0,4) accepted; (1,1) (1,1) rejected
Which number formats are allowed?	Whole numbers with an optional minus sign. No + sign, decimals or non-ASCII digits.	+1, 1.5, ² and an empty number are each rejected
Are commands case-sensitive?	Yes: add, mod, rm and gg in lower case only.	ADD is an unknown command

7. Testing

There are two test scripts in the `tests` folder. Neither changes the program; they run it and check what it prints.

- **tests/check_sample.py** runs the assignment's sample and compares each graph with the expected output by coordinates, so vertex numbering may differ. All three graphs pass.
- **tests/test_a1.py** is the full suite: 72 tests in five groups, listed below. Expected results were worked out by hand from the assignment's rules, not copied from the program's output.

```
python3 tests/test_a1.py                 # test a1/ece650-a1.py
python3 tests/test_a1.py other/file.py   # test any other version
```

Group	Tests	What is checked
1. Sample	6	The three sample graphs; no errors; vertices keep their numbers after mod.
2. Graph cases	10	Non-whole intersection; overlap; touching at an end-point; streets that never meet; parallel streets; extra whitespace; a segment with no crossing; three streets through one point; blank lines; rm then gg.
3. Assumptions	24	The rows of the table in section 6 (number formats and command case are covered in group 4).
4. Error messages	27	Every error the program can give, each followed by a valid gg to show the program kept going.
5. Robustness	5	Errors only on standard error; carrying on after errors; a failed add or mod changes nothing; 30 crossing streets; empty input.

Result: all 72 tests pass.

Checking that the tests can fail

A test suite that always passes proves nothing, so it was also run against three copies of the program with a deliberate bug each:

Deliberate bug	Tests that caught it
Always print two decimals, e.g. (2.00,2.00)	16 failed
Count a street crossing itself as an intersection	4 failed
Join neighbouring vertices even when neither is an intersection	1 failed

Other checks

- **Regression check.** When the parser was rewritten without regular expressions, both versions were run on the same 34-line input: their graph output was identical and they rejected the same 21 lines.
- **Random comparison.** 1,000 random inputs were run through this program and an independently written one (see the separate test report). The graphs matched every time.

Error messages

Every message the program can produce, with an input that triggers it. Each row is a test in group 4.

Input	Message after “Error: ”
`add "A" ...` when A exists	`add' specified for a street that already exists
`mod/rm "X"` when X doesn't exist	`mod' or `rm' specified for a street that does not exist
`add "B" (1,1)` or `add "B"`	a street needs at least two coordinates
`add "B" (1,1) (1,1)`	consecutive coordinates of a street must differ
`add "B"(0,4) (4,0)`, `(0,4)(4,0)`	coordinates must be separated by spaces
`add B (0,4) (4,0)`	street name must be in double quotes
`add "B (0,4) (4,0)`	missing closing quote on street name
`add "B1" ...`	street name may only contain letters and spaces
`add "" ...` or `" "`	street name cannot be empty
`add "B" 0,4 4,0`	expected '(' at the start of a coordinate
`add "B" (0,4) (4,0`	missing ')' in a coordinate
`(0,4,1)` or `(0)`	a coordinate must look like (x,y)
`(1.5,2)`, `(+1,4)`, `(²,4)`, `(,4)`	coordinates must be integers
`add` or `rm` with nothing after	expected a space before the street name
`rm "A" extra`	`rm' takes only a street name
`gg extra`	`gg' takes no arguments
`foo`, `ADD`, `add"B"`	unknown command `<word>', e.g. unknown command `foo'

8. Alternative approaches

Regular expressions for parsing

A **regular expression** (regex) is a compact pattern language for matching text. The whole valid add line can be described by a single pattern, roughly `^\s*(add|mod)\s+"([A-Za-z ]+)"(\s+\(\s*-?\d+\s*,\s*-?\d+\s*\))+\s*$`, where `\s` means whitespace, `\d` a digit and `+` “one or more”. It is much shorter and anything that doesn't match is rejected in one go. The downsides are that it is hard to read for anyone not fluent in regex, and a failed match only says “the line is wrong”, not *what* is wrong. The first version of this program used regex; the current one uses plain string operations so each step is visible and each error is specific.

Floating-point arithmetic instead of Fractions

The common approach is to compute intersections with floats and compare points with a small tolerance, e.g. round to 2 decimals or treat |a – b| < 1e-9 as equal. It is slightly faster and simpler to print, but the same intersection reached from different segment pairs may differ in the last bits (2.6), which can create duplicate vertices or missed edges. Fractions avoid that at a small cost in speed.

Sweep-line algorithm (Bentley–Ottmann)

Testing every pair of segments is $O(n^2)$. A sweep line moves across the plane and only tests segments that are next to each other in the sweep order, finding all k intersections in $O((n + k) \log n)$. It is much faster for thousands of streets but far harder to write and debug, especially with vertical or overlapping segments. For assignment-sized inputs the simple approach is the better trade-off.

Incremental graph updates

Instead of rebuilding on every gg, the graph could be updated as streets change: only the changed street needs testing against the others. This is faster if gg is called very often, but removing a street then means working out which vertices and edges still belong to the remaining streets. Rebuilding is simpler and clearly correct.

Renumbering vertices on every gg

Vertices could be numbered 1, 2, 3, ... afresh for each gg. That gives tidier output, but the sample keeps numbers stable across commands, and stable numbers make output easier to compare between steps. Reusing the numbers of deleted vertices is another option; the spec doesn't require it.

Object-oriented geometry

Points and segments could be classes (`Point`, `Segment`) with methods such as `seg.intersect(other)`. That reads well and scales to bigger programs, but plain tuples are hashable for free (2.3), which is exactly what the dictionary of vertex IDs needs. For a program this size, functions on tuples are enough.

Graph library

A library such as networkx could store the graph, but it may not be installed on the marking servers and the assignment forbids installing packages. The graph here is only printed, so a list of edges is all that's needed.